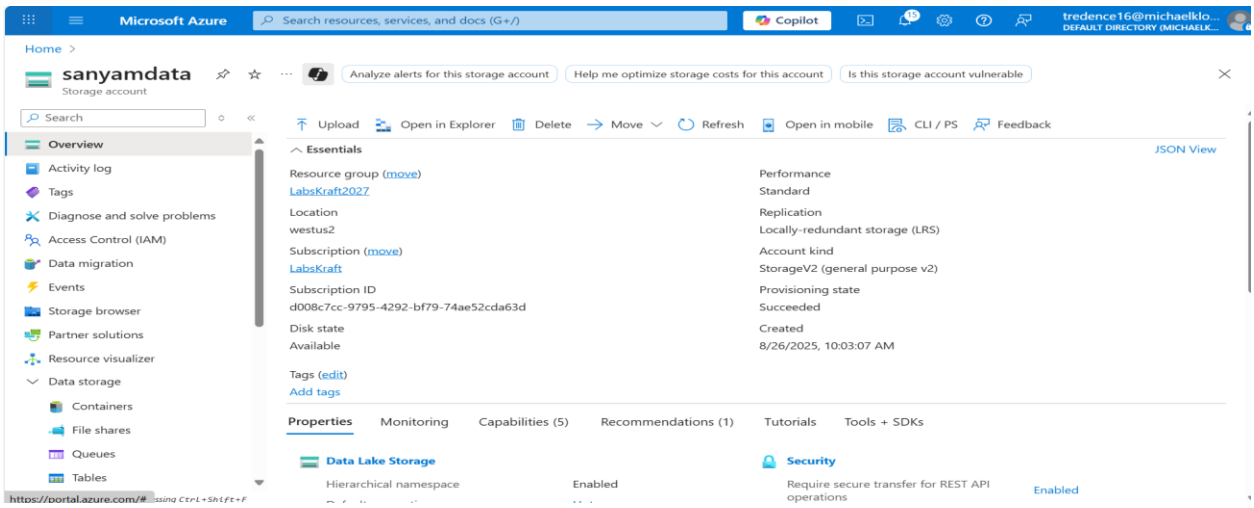
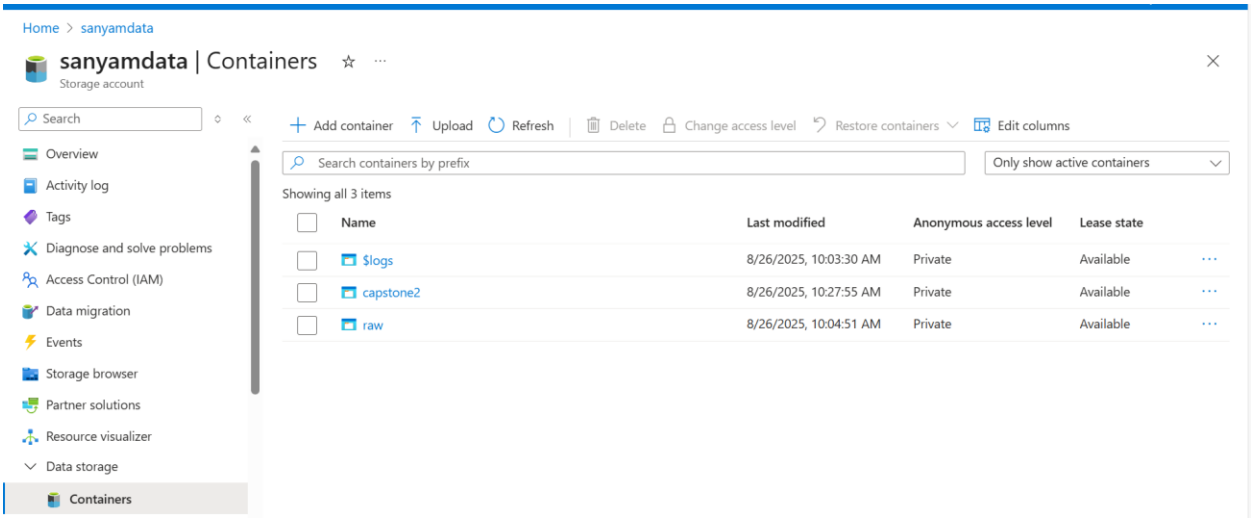
SANYAM ARORA – CAPSTONE-2-REPORT

Step-1: I have created a storage account named ‘sanyamdata’, with Azure Data Lake Storage Gen2 enabled as the primary service, and hierarchical namespace support activated.

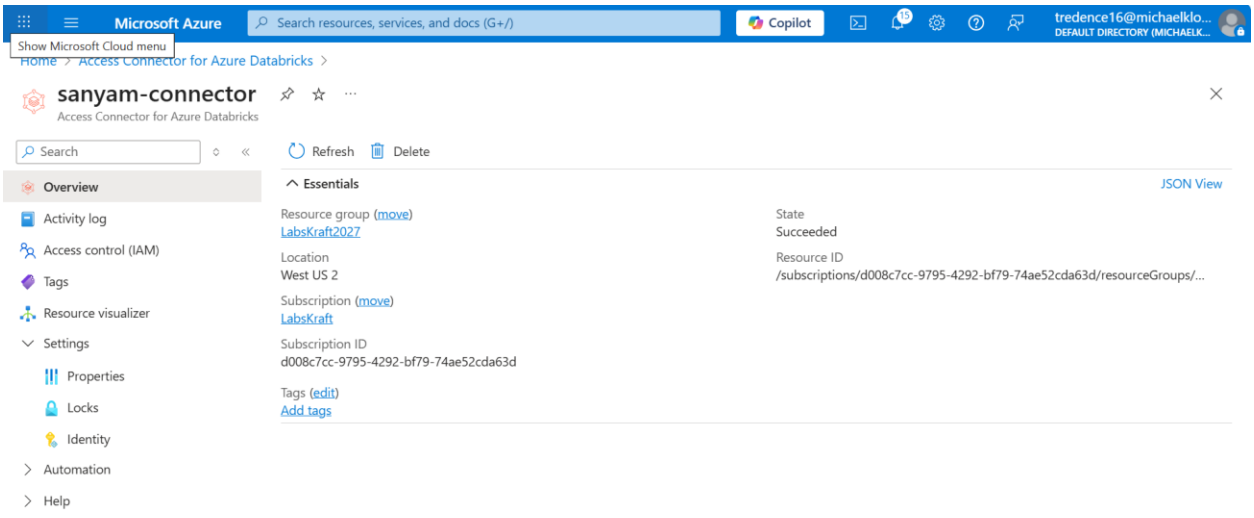


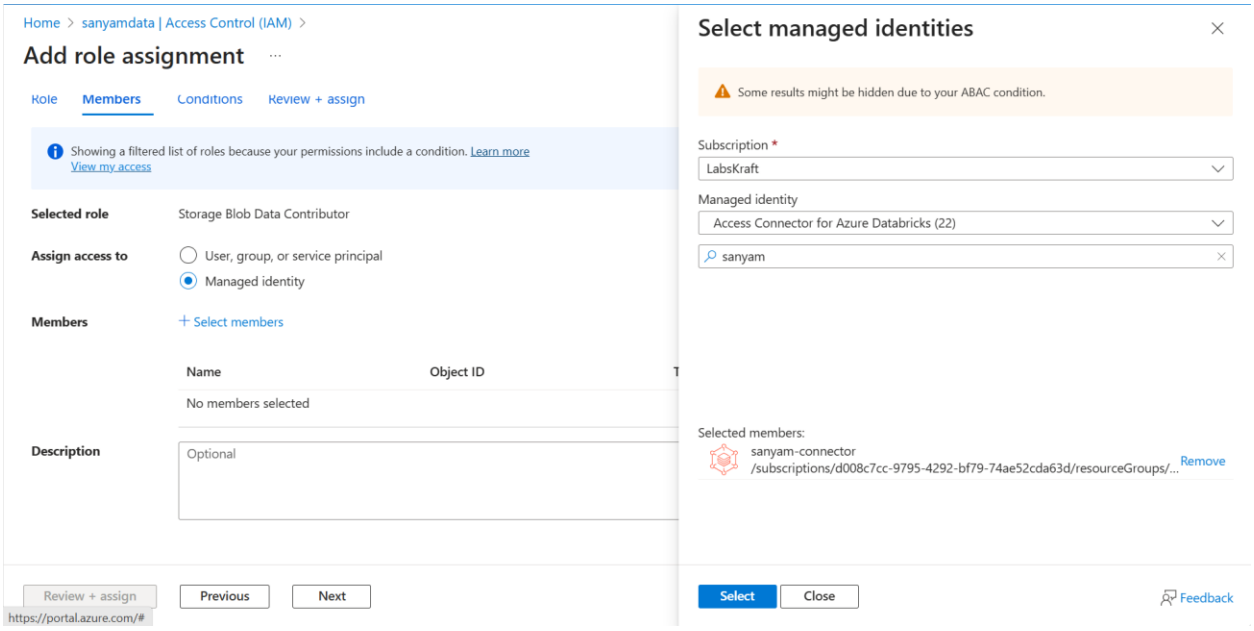
Step-2: : I have added a container named ‘capstone2’ in the storage account, and within it, I created a directory called ‘sales’ and ‘products’. Inside ‘sales’ and ‘products’, I have ingested a CSV files of both the folders.



Step 3: I configured **IAM permissions** on my Azure Storage Account (sanyamdata) by creating **Databricks connector** and assigning the Storage Blob Data Contributor role to the managed identity used by Databricks.

Purpose: I assigned the **Storage Blob Data Contributor** role to Databricks' managed identity to ensure secure and authorized access to my data lake. This is a critical step for enabling Delta Live Tables to read, write, and manage data across the Bronze, Silver, and Gold layers in the medallion architecture, while maintaining compliance with Unity Catalog's access and governance requirements.





Step-4: I created a **Unity Catalog** named “Sanyam_capstone2_catalog” to manage data governance across my Databricks workspace. Before creating the catalog, I set up an external location named “Sanyam_capstone2_ext_loc”, which points to my Azure Data Lake Storage Gen2 (sanyamdata).

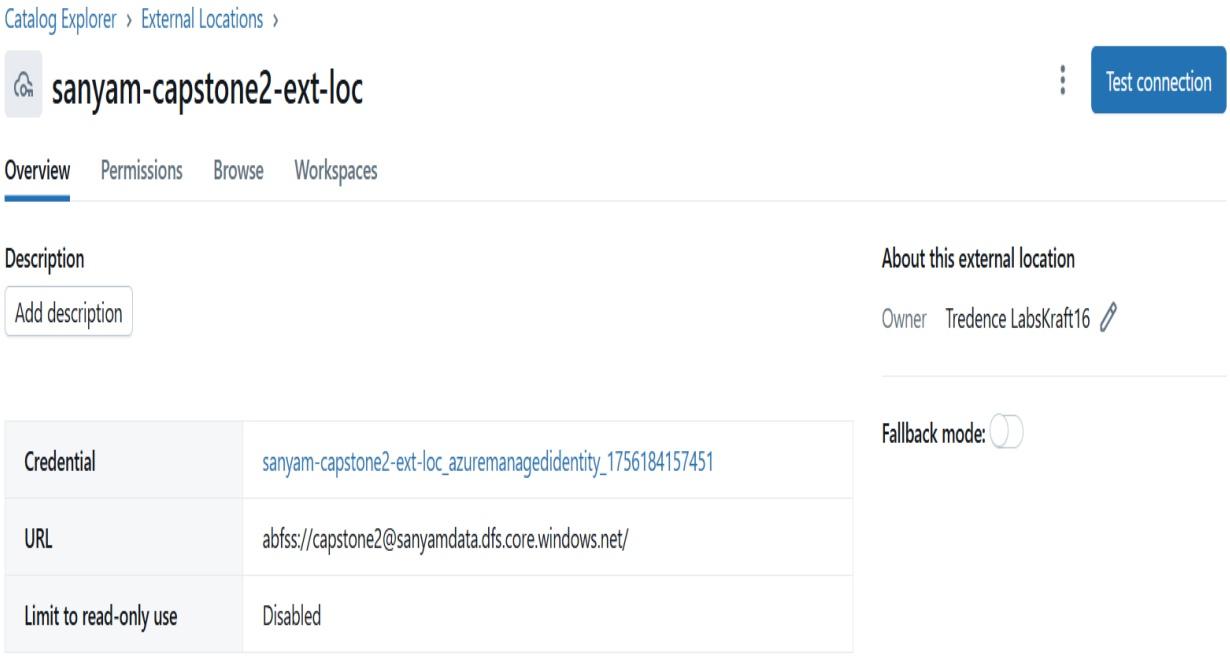
Purpose: I created an external location before setting up the Unity Catalog to securely link my Azure Data Lake Storage Gen2 with Databricks. This step is essential for enabling Unity Catalog to manage access, enforce governance, and organize data across the Bronze, Silver, and Gold layers of the medallion architecture. The external location acts as the base path for organizing your Bronze, Silver, and Gold layers.

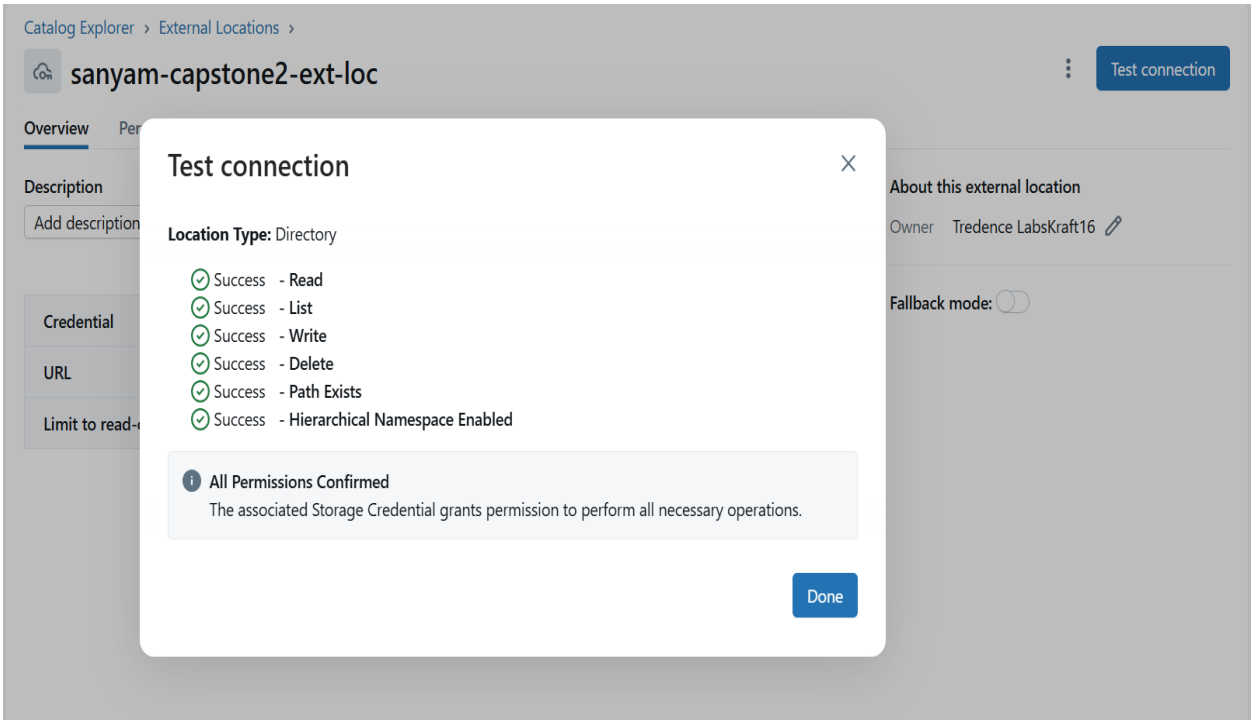
It enables:

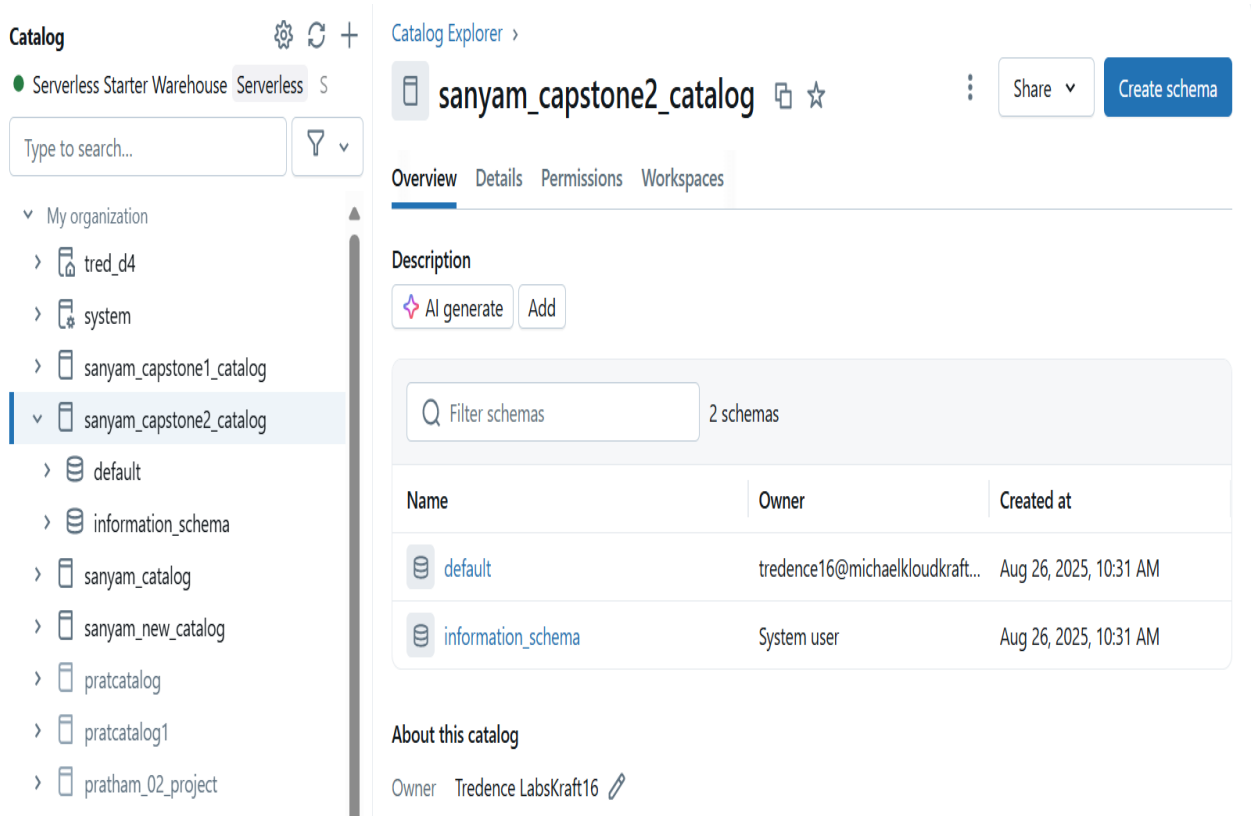
• Structured data management.

• Schema enforcement and lineage tracking.

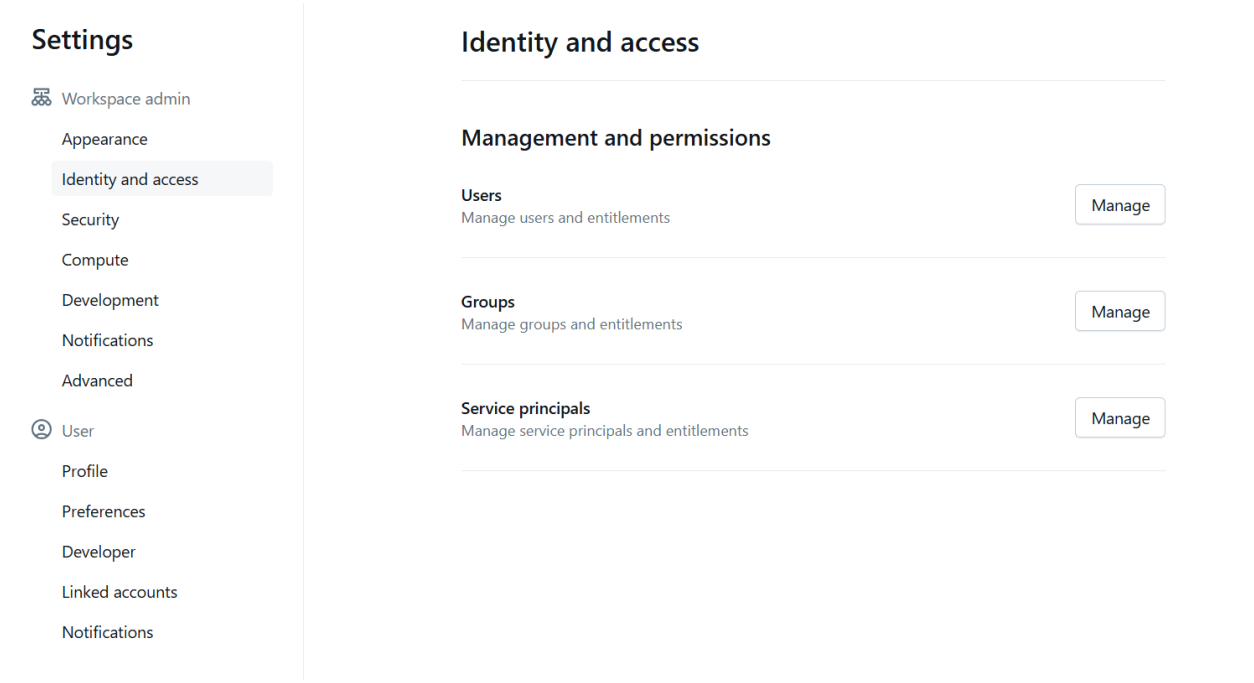
• Integration with Auto Loader and Delta Live Tables.

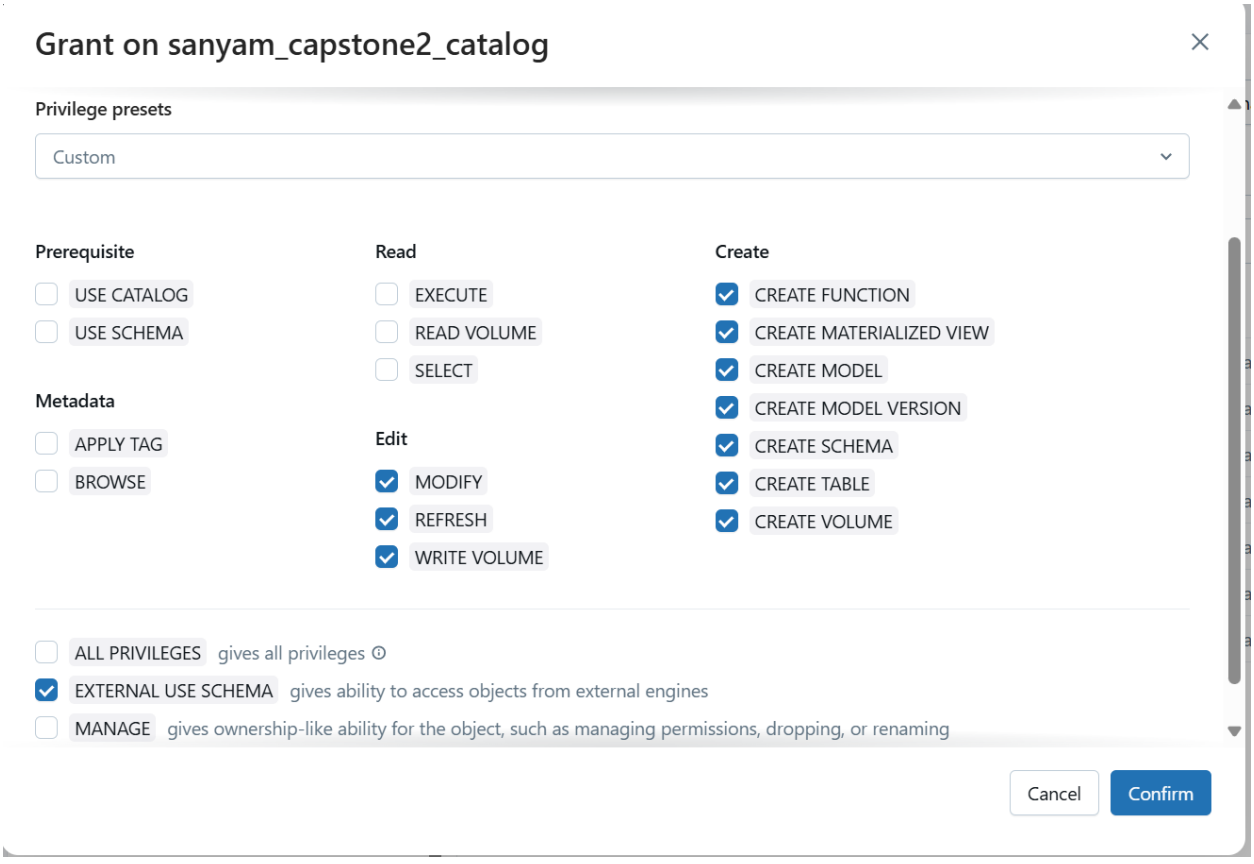


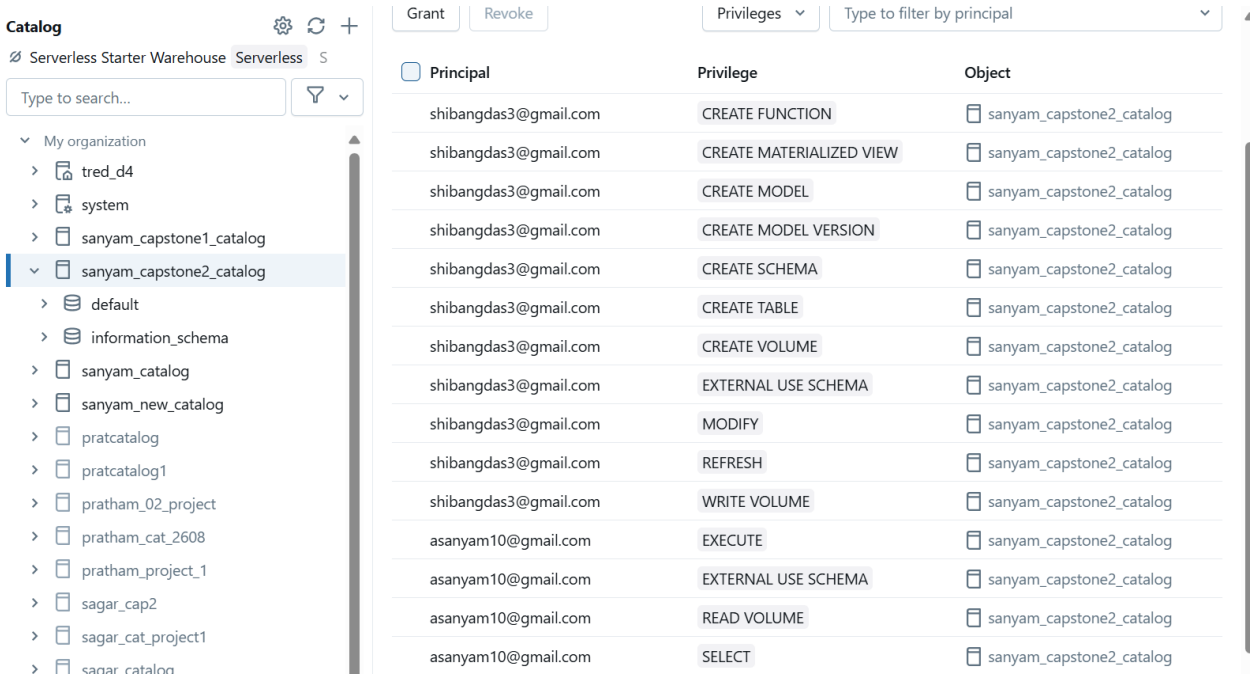

Catalog Explorer › External Locations ›
sanyam-capstone2-ext-loc
Test connection
Overview
Description
Add description
Credential
URL
Limit to read-
About this external location
Owner Tredence LabsKraft16
Fallback mode:
Test connection
Location Type: Directory
Success - Read
Success - List
Success - Write
Success - Delete
Success - Path Exists
Success - Hierarchical Namespace Enabled
All Permissions Confirmed
The associated Storage Credential grants permission to perform all necessary operations.
Done


Catalog
Serverless Starter Warehouse Serverless
Type to search...
My organization
tred_d4
system
sanyam_capstone1_catalog
sanyam_capstone2_catalog
default
information_schema
sanyam_catalog
sanyam_new_catalog
pratcatalog
pratcatalog1
pratham_02_project
Catalog Explorer ›
sanyam_capstone2_catalog
Share
Create schema
Overview
Details
Permissions
Workspaces
Description
AI generate
Add
Filter schemas
2 schemas
Name
Owner
Created at
default
tredence16@michaelkloudkraft...
Aug 26, 2025, 10:31 AM
information_schema
System user
Aug 26, 2025, 10:31 AM
About this catalog
Owner Tredence LabsKraft16

Step-5: I have given the access to users for read, edit and create. I have given the read access to ‘asanyam10@gmail.com’ and Create and Edit access to ‘shibangdas3@gmail.com’.







CREATE A NEW SCHEMA

`%sql

USE CATALOG sanyam_capstone2_catalog;

CREATE SCHEMA IF NOT EXISTS sanyam_capstone2_db;

USE SCHEMA sanyam_capstone2_db;

CODE:

```
import dlt
from pyspark.sql.functions import *
@dlt.table(
 name="sales_raw",
 comment="Raw sales data ingested from CSV files",
 table_properties={"quality": "bronze"}
)
def sales_raw():
   return (
      spark.readStream.format("cloudFiles")    # Auto Loader for incremental loads
```

```
      .option("cloudFiles.format", "csv")
      .option("header", "true")
      .load("abfss://capstone2@sanyamdata.dfs.core.windows.net/sales/")
  )

# Products CSV
@dlt.table(
 name="products_raw_csv",
 comment="Raw product data from CSV",
 table_properties={"quality": "bronze"}
)
def products_raw_csv():
  return (
    spark.readStream.format("cloudFiles")
    .option("cloudFiles.format", "csv")
    .option("header", "true")
    .load("abfss://capstone2@sanyamdata.dfs.core.windows.net/products/")
  )

# Products JSON
@dlt.table(
 name="products_raw_json",
 comment="Raw product data from JSON",
 table_properties={"quality": "bronze"}
)
def products_raw_json():
  return (
    spark.readStream.format("cloudFiles")
    .option("cloudFiles.format", "json")
    .load("abfss://capstone2@sanyamdata.dfs.core.windows.net/products/")
  )
```

```
# Unified Products (CSV + JSON)
@dlt.view(name="products_raw")
def products_raw():
   return dlt.read("products_raw_csv").unionByName(dlt.read("products_raw_json"), allowMissingColumns=True)


# Clean Sales
@dlt.table(
 name="sales_clean",
 comment="Cleaned sales data with enforced schema",
 table_properties={"quality": "silver"}
)
@dlt.expect("valid_sales_amount", "sales_amount >= 0")
@dlt.expect("valid_quantity", "quantity > 0")
def sales_clean():
   return (
      dlt.read("sales_raw")
      .withColumn("sale_id", col("sale_id").cast("string"))
      .withColumn("product_id", col("product_id").cast("string"))
      .withColumn("region", col("region").cast("string"))
      .withColumn("store_id", col("store_id").cast("string"))
      .withColumn("quantity", col("quantity").cast("int"))
      .withColumn("sales_amount", col("sales_amount").cast("double"))
      .withColumn("sale_date", to_date(col("sale_date"), "yyyy-MM-dd"))
      .na.drop()
   )


@dlt.table(
 name="products_clean",
 comment="Cleaned products data with schema evolution support",
 table_properties={"quality": "silver"}
```

```
)
def products_clean():
  return (
    dlt.read("products_raw")
    .withColumn("product_id", col("product_id").cast("string"))
    .withColumn("product_name", col("product_name").cast("string"))
    .withColumn("category", col("category").cast("string"))
    .withColumn("price", col("price").cast("double"))
    .withColumn("discount_rate", col("discount_rate").cast("double"))
  )

# Join sales + products and calculate KPIs
@dlt.table(
 name="sales_summary",
 comment="Daily sales summary by product and region",
 table_properties={"quality": "gold"}
)
def sales_summary():
  sales = dlt.read("sales_clean")
  products = dlt.read("products_clean")
  return (
    sales.join(products, "product_id", "left")
    .groupBy("region", "sale_date", "product_id", "product_name", "category")
    .agg(
      sum("sales_amount").alias("daily_revenue"),
      sum("quantity").alias("units_sold"),
      avg("price").alias("avg_price"),
      avg("discount_rate").alias("avg_discount")
    )
  )
```

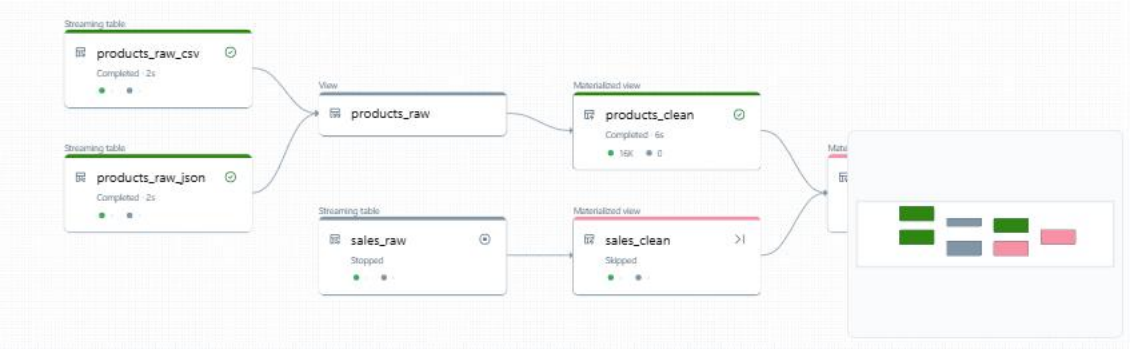
Correct pipeline

Once it is done, we will create a change in data to see whether our pipeline detects error in it or not.

I have created a dummy file named: 'sales_data' and added one extra column (clone_id) to it .

sale_id	product_id	region	store_id	quantity	sales_amount	sale_date	clone_id
1	157	APAC	Store_31	16	1289.03	8/11/2025 21:20	123
2	65	APAC	Store_9	8	1301.48	8/12/2025 2:40	-1
3	416	LATAM	Store_25	19	3047.56	8/10/2025 0:09	abc
4	501	US	Store_29	18	1955.74	7/28/2025 9:49	string
5	244	MEA	Store_6	20	2362.98	8/15/2025 0:28	0
6	240	EU	Store_22	8	1516.96	8/25/2025 5:58	0
7	854	EU	Store_24	10	178.92	7/28/2025 9:55	-9
8	69	US	Store_9	7	246.4	8/10/2025 22:41	
9	654	EU	Store_3	14	138.23	8/2/2025 5:50	
10	277	US	Store_8	15	145.13	8/15/2025 19:05	
11	584	LATAM	Store_4	13	2516.03	8/8/2025 13:20	
12	432	APAC	Store_1	13	241.39	7/27/2025 22:24	
13	797	US	Store_25	11	1914.19	8/5/2025 0:57	
14	64	MEA	Store_32	15	2545.81	8/9/2025 6:41	
15	256	US	Store_15	18	1482.75	7/31/2025 9:45	
16	924	LATAM	Store_14	7	670.92	8/17/2025 22:33	

Error:



Pipeline event log details

This event is associated with `sanyam_capstone2_catalog.sanyam_capstone2_db.sales_raw` (Streaming table).

Summary JSON

WARN 2025-08-26 14:08:54 IST flow_progress

Flow 'sanyam_capstone2_catalog.sanyam_capstone2_db.sales_raw' has encountered a schema change during execution and terminated. A new update using the new schema will be automatically started.

Error details

```
com.databricks.pipelines.common.errors.DLTSparkException: [FLOW_SCHEMA_CHANGED] Flow sanyam_capstone2_catalo
g.sanyam_capstone2_db.sales_raw has terminated since it encountered a schema change during execution.
The schema change is compatible with existing target schema and the next run of the flow can resume with the new
schema.
org.apache.spark.sql.streaming.StreamingQueryException: [STREAM_FAILED] Query [id = 727283e8-5ff4-40a5-a4f0-8
01aa8a110d1, runId = d8417ae9-49b0-4f45-893a-92abe132caf6] terminated with exception: [UNKNOWN_FIELD_EXCEPTION.NE
W_FIELDS_IN_FILE] Encountered unknown fields during parsing: [clone_id], which can be fixed by an automatic retr
y: true
 Unknown fields inside file path: abfss://capstone2@sanyamdata.dfs.core.windows.net/sales/sales_data.csv
Rescued file schema: clone_id STRING SQLSTATE: KD003 SQLSTATE: XXKST
=== Streaming Query ===
Identifier: sanyam_capstone2_catalog.sanyam_capstone2_db.__materialization_mat_544eb7bf_bd42_4dbd_b70b_94072b414d
37_sales_raw_1 [id = 727283e8-5ff4-40a5-a4f0-801aa8a110d1, runId = d8417ae9-49b0-4f45-893a-92abe132caf6]
```

The streaming flow sales_raw failed due to a schema change — a new field clone_id was found in the incoming data (sales_data.csv) that wasn't expected. This caused the flow to terminate, but since the change is compatible, it can be resolved by an automatic retry with the updated schema.

CODE FOR ERROR CORRECTION:

```
import dlt
from pyspark.sql.functions import *
@dlt.table(
  name="sales_raw",
  comment="Raw sales data ingested from CSV files",
  table_properties={"quality": "bronze"}
)
def sales_raw():
    return (
       spark.readStream.format("cloudFiles")     # Auto Loader for incremental loads
       .option("cloudFiles.format", "csv")
       .option("header", "true")
       .load("abfss://capstone2@sanyamdata.dfs.core.windows.net/sales/")
        # Force clone_id column to integer type regardless of source type
       .withColumn("clone_id", col("clone_id").cast("int"))
    )

# Products CSV
@dlt.table(
  name="products_raw_csv",
  comment="Raw product data from CSV",
  table_properties={"quality": "bronze"}
)
def products_raw_csv():
    return (
       spark.readStream.format("cloudFiles")
       .option("cloudFiles.format", "csv")
       .option("header", "true")
       .load("abfss://capstone2@sanyamdata.dfs.core.windows.net/products/")
    )
```

```
# Products JSON
@dlt.table(
 name="products_raw_json",
 comment="Raw product data from JSON",
 table_properties={"quality": "bronze"}
)
def products_raw_json():
   return (
      spark.readStream.format("cloudFiles")
      .option("cloudFiles.format", "json")
      .load("abfss://capstone2@sanyamdata.dfs.core.windows.net/products/")
   )


# Unified Products (CSV + JSON)
@dlt.view(name="products_raw")
def products_raw():
   return dlt.read("products_raw_csv").unionByName(dlt.read("products_raw_json"), allowMissingColumns=True)


# Clean Sales
@dlt.table(
 name="sales_clean",
 comment="Cleaned sales data with enforced schema",
 table_properties={"quality": "silver"}
)
@dlt.expect("valid_sales_amount", "sales_amount >= 0")
@dlt.expect("valid_quantity", "quantity > 0")
def sales_clean():
   return (
      dlt.read("sales_raw")
      .withColumn("sale_id", col("sale_id").cast("string"))
```

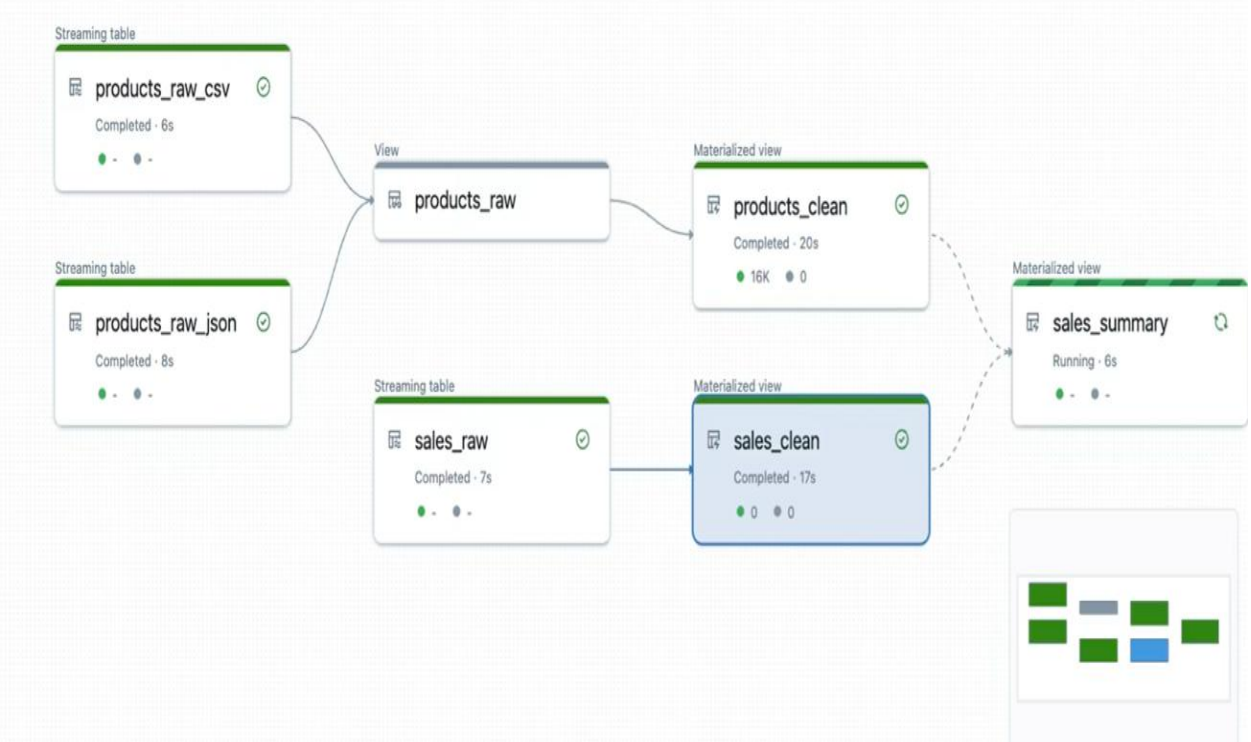
```
      .withColumn("product_id", col("product_id").cast("string"))
      .withColumn("region", col("region").cast("string"))
      .withColumn("store_id", col("store_id").cast("string"))
      .withColumn("quantity", col("quantity").cast("int"))
      .withColumn("sales_amount", col("sales_amount").cast("double"))
      .withColumn("sale_date", to_date(col("sale_date"), "yyyy-MM-dd"))
      .na.drop()
   )

@dlt.table(
 name="products_clean",
 comment="Cleaned products data with schema evolution support",
 table_properties={"quality": "silver"}
)
def products_clean():
   return (
      dlt.read("products_raw")
      .withColumn("product_id", col("product_id").cast("string"))
      .withColumn("product_name", col("product_name").cast("string"))
      .withColumn("category", col("category").cast("string"))
      .withColumn("price", col("price").cast("double"))
      .withColumn("discount_rate", col("discount_rate").cast("double"))
   )

# Join sales + products and calculate KPIs
@dlt.table(
 name="sales_summary",
 comment="Daily sales summary by product and region",
 table_properties={"quality": "gold"}
)
def sales_summary():
```

```
sales = dlt.read("sales_clean")
products = dlt.read("products_clean")
return (
  sales.join(products, "product_id", "left")
  .groupBy("region", "sale_date", "product_id", "product_name", "category")
  .agg(
    sum("sales_amount").alias("daily_revenue"),
    sum("quantity").alias("units_sold"),
    avg("price").alias("avg_price"),
    avg("discount_rate").alias("avg_discount")
  )
)
```

Pipeline after Correction:



Time Travel:

CODE:

1. Listed tables in our catalog.

   %sql

   SHOW TABLES IN sanyam_capstone2_catalog.sanyam_capstone2_db;

2. Created a new Delta table (`sales_clean_tbl`) from an existing one.

   ```
   CREATE OR REPLACE TABLE
   sanyam_capstone2_catalog.sanyam_capstone2_db.sales_clean_tbl AS SELECT *
   FROM sanyam_capstone2_catalog.sanyam_capstone2_db.sales_clean;
   ```

3. Viewed the table history using `DESCRIBE HISTORY`.

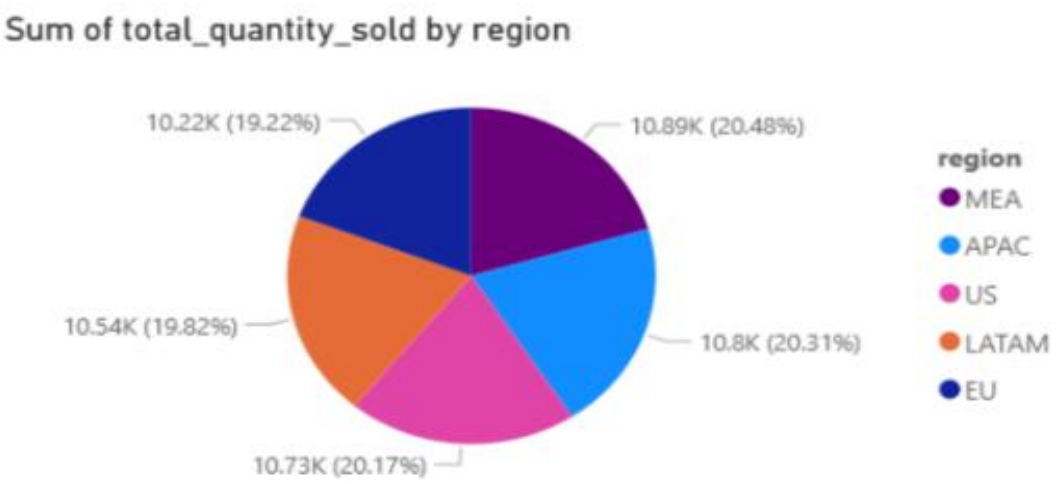
   DESCRIBE HISTORY
   sanyam_capstone2_catalog.sanyam_capstone2_db.sales_clean_tbl;

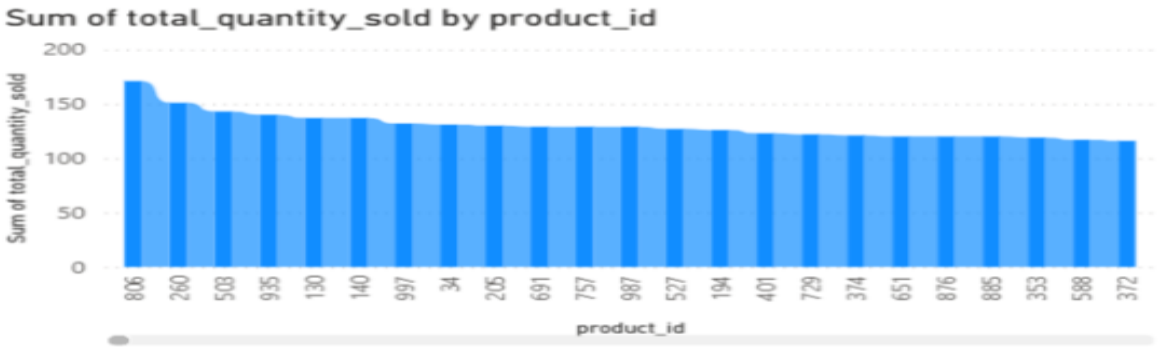
_sqldf: pyspark.sql.connect.dataframe.DataFrame = [database: string, tableName: string ... 1 more field]

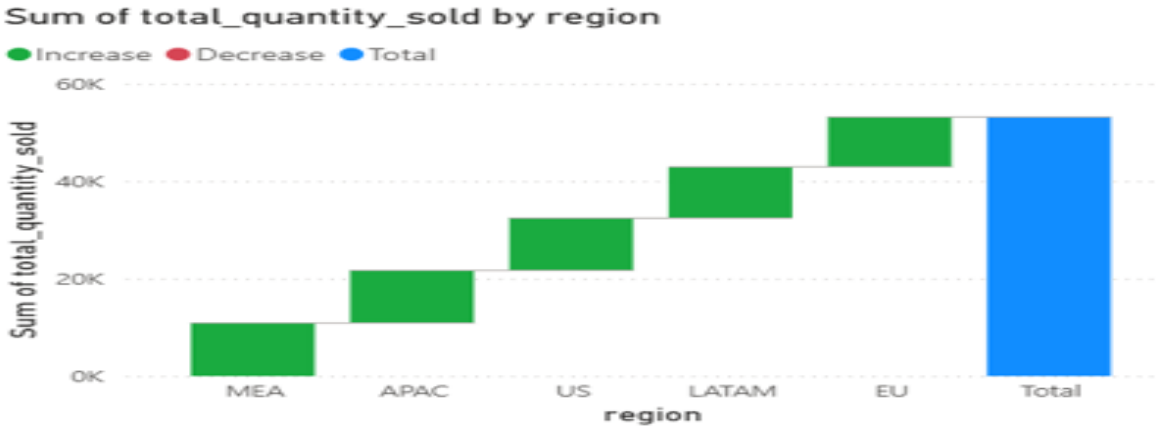
Table +

	database	tableName	isTemporary
1	sanyam_capstone2_...	abc	false
2	sanyam_capstone2_...	bronze_products_csv	false
3	sanyam_capstone2_...	bronze_products_js...	false
4	sanyam_capstone2_...	bronze_sales	false
5	sanyam_capstone2_...	event_log_sanyam	false
6	sanyam_capstone2_...	gold_sales_summary	false
7	sanyam_capstone2_...	products_clean	false
8	sanyam_capstone2_...	products_raw_csv	false
9	sanyam_capstone2_...	products_raw_json	false
10	sanyam_capstone2_...	sales_clean	false
11	sanyam_capstone2_...	sales_raw	false
12	sanyam_capstone2_...	sales_summary	false
13	sanyam_capstone2_...	silver_products	false
14	sanyam_capstone2_...	silver_sales	false

Power BI Output of the Pipeline

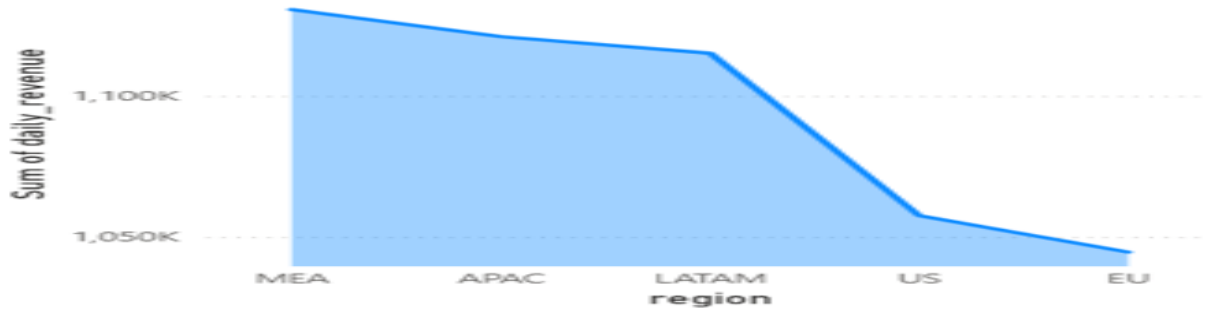






Sum of total_quantity_sold by region

Sum of daily_revenue by region



Sum of daily_revenue by region